

Linux Services

D. Leeuw

10 december 2025

v.0.4.2



© 2021-2025 Dennis Leeuw

Dit werk is uitgegeven onder de Creative Commons BY-NC-SA Licentie en laat anderen toe het werk te kopiëren, distribueren, vertonen, op te voeren, en om afgeleid materiaal te maken, zolang de auteurs en uitgever worden vermeld als maker van het werk, het werk niet commercieel gebruikt wordt en afgeleide werken onder identieke voorwaarden worden verspreid.

Over dit Document

Inhoudsopgave

Over dit Document	i
1 Booting Linux en /boot/	1
1.1 GRUB Configuratie	1
1.2 Kernel boot bestanden	2
2 Werken met daemons	5
2.1 Het eerste proces	5
2.2 Unit files	6
2.3 systemctl	7
3 Job scheduling	9
3.1 cron	9
3.2 at	10
4 Monitoring services	11
4.1 Proces monitoring	11
4.1.1 top	12
4.2 Daemons en netwerken	13
Index	17

Hoofdstuk 1

Booting Linux en `/boot/`

Nadat het BIOS klaar is met zijn taken wil deze een boot-loader laden. Vroeger was dit LiLo, de Linux Loader, maar tegenwoordig wordt GRUB, GRand Unified Bootloader, gebruikt. GRUB kan verschillende operating systems laden en kan dus gebruikt worden als universele bootloader om zowel Linux als Windows te starten. Zo kan je op één machine Windows en Linux hebben staan en kan je door te herstarten kiezen welk OS je wil gebruiken.

De bootloader laadt de kernel, Linux, welke ervoor zorgt dat er tegen de hardware gesproken kan worden. De taken van de kernel zijn:

- het beheren van het fysieke geheugen en daarbij behorend het opdelen van het virtuele geheugen voor proces isolatie
- zorgen dat zowel processen als hardware de resources krijgen waarom ze vragen (interrupts), als dat mag
- het serialiseren van de taken die uitgevoerd moeten worden, ofwel proces management (schedulig)

Als de kernel zijn taken verder op de achtergrond kan uitvoeren kan proces nummer 1 gestart worden om het verdere bootproces af te maken, meestal eindigend in een grafische of een command line interface, waarna de gebruiker(s) aanzet zijn.

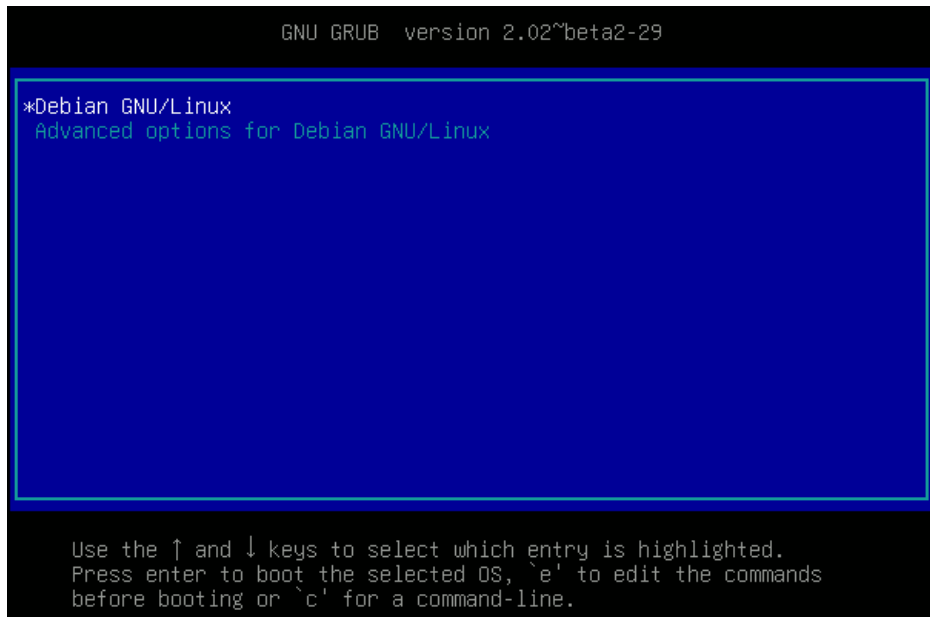
Als je een `ls` doet van de `/boot/` directory dan vinden we daarin de belangrijkste bestanden die nodig zijn om een Linux systeem op te kunnen starten.

1.1 GRUB Configuratie

De configuratie van GRUB staat in `/etc/` en in `/boot/grub/` (soms in `/boot/grub2/` of een ander versienummer). De GRUB bestanden in `/boot/`

zijn gegenereerde bestanden en daar zou je nooit zelf wijzigingen in moeten aanbrengen. De wijzigingen worden aangebracht in `/etc/default/grub` en de snippets in `/etc/grub.d`. Een script `grub-mkconfig` maakt van dit geheel de configuratie in `/boot/`.

Tijdens het opstarten laat GRUB weten dat het de bootloader is en krijg je de mogelijkheid om wijzigingen in het bootproces aan te brengen. GRUB geeft in een menu weer welke standaard opstart opties het kent, zoals je kan zien in [1.1](#)



Figuur 1.1: GRUB bootscherm

Onder aan het scherm zie je hoe je GRUB kan bedienen (met de pijltjes toetsen) en dat het met `e` de opdracht regel kan editten (wijzigen). Dit laatste zal je niet vaak nodig hebben, maar weet dat het kan.

1.2 Kernel boot bestanden

De kernel, Linux, bestaat uit een modulaire kernel. Er is een stuk basis software die geladen wordt tijdens het opstarten en deze software kan uitgebreid worden met modules. Modules voegen extra functionaliteit toe, zoals bij voorbeeld netwerk-drivers. De reden dat er gekozen voor een modulair design is dat de basis kernel daardoor klein kan zijn en minder geheugen inneemt. Alleen de drivers of andere functionaliteit die echt nodig is wordt geladen.

De basis kernel ook bekend als `vmlinuz` kan gevonden worden in de `/boot/` directory, in het verleden werd de kernel in de `/` directory gezet en kan daar op sommige systemen nog gevonden worden. In de `/boot/` directory kan je ook bestanden vinden die beginnen met `vmlinuz` en waarna er een serienummer volgt, zo kunnen er meerdere versies van de kernel staan, waaruit eventueel via de bootloader gekozen kan worden.

Naast het bestand `vmlinuz` staan er nog een paar bestanden die bij de kernel horen:

config is de configuratie van de kernel zoals deze gebruikt is tijdens het compilen van de kernel. Het beschrijft o.a. welke stukken van de kernel als module gebouwd moeten worden. Dit document is niet belangrijk voor het opstarten van het systeem.

initrd.img Ook de kernel heeft tijdens het opstarten modules nodig, denk daarbij aan de netwerk-drivers. De `initrd.img` bevat deze drivers en deze image wordt in RAM geladen zodat de modules snel beschikbaar zijn (ipv. dat ze van een trage harddisk moeten komen)

System.map bevat een overzicht van welke functies of variabelen zich op welke lokatie bevinden in de kernel. Dit bestand wordt gebruikt om lookups te doen als er een kernel oops of kernel panic zich voordoet.

Als de kernel klaar is met laden, dan moet deze de controle overgeven aan wat vroeger `init` heette. Dit was het eerste proces dat opgestart werd en `init` was verantwoordelijk voor het opstarten van andere processen, waaronder het login-proces. Tegenwoordig is op de meeste GNU/Linux systemen de functie van `init` overgenomen door `systemd`.

Hoofdstuk 2

Werken met daemons

In de Unix wereld zeggen we altijd 'Everything is a file and if it is not a file it is a process'.

Processen die op de achtergrond draaien heten daemons in de Unix-wereld. Ze zijn ook bekend als services, zeker in de Windows wereld. Daemons zijn dus processen waar je weinig van merkt en die rustig hun werk staan te doen. Deze processen draaien zonder dat ze input van de gebruiker nodig hebben.

Bekende daemons zijn bijvoorbeeld web- of mailservers, maar het kan ook een printserver zijn. Op een Linux-machine, ongeacht of het een desktop of een server is, draaien verschillende daemons in de achtergrond.

2.1 Het eerste proces

Nadat de kernel in het geheugen geladen is zal deze gestart worden. De Linux kernel zorgt ervoor dat de beschikbare hardware klaar is voor gebruik en dat er processen gestart kunnen worden. Het eerste proces dat de kernel start is **systemd**. Vroeger was dit **init** en dat kan je op vele ander Unix-achtige besturingssystemen nog tegen komen, maar de meeste Linux distributies zijn over naar **systemd**.

systemd is het proces dat ervoor zorgt dat alle ander processen gestart worden. De **systemd** daemon heeft proces nummer 1. Het is de eerste daemon die start bij het opstarten van het systeem en de laatste die afgesloten wordt bij het afsluiten van het systeem.

Het proces met nummer 1 wordt ook wel "The mother of all processes" genoemd. Nadat de kernel klaar is met opstarten tijdens het boot-proces start het dit eerste proces op. Proces nummer 1 is daarna verantwoordelijk voor het verdere opstart proces van het systeem. Het is dan ook proces 1

die bepaalt of er bijvoorbeeld een grafische interface wordt gestart, maar ook welke functies een machine kan aanbieden nadat het opgestart is. Is de machine een mail-server of een web-server, of misschien wel helemaal geen server.

2.2 Unit files

Om een service te starten of te stoppen gebruikt **systemd** bestanden waarin beschreven wordt hoe een service heet en welke commando's er nodig zijn om te starten en stoppen. Deze beschrijving van een service wordt binnen **systemd** een **unit** genoemd. Units kan je onder andere terug vinden in de `/etc/systemd/system` directory. Een voorbeeld van een unit-file in Debian 11 is de `syslog.service`. Deze ziet er zo uit:

```
[Unit]
Description=System Logging Service
Requires=syslog.socket
Documentation=man:rsyslogd(8)
Documentation=man:rsyslog.conf(5)
Documentation=https://www.rsyslog.com/doc/

[Service]
Type=notify
ExecStart=/usr/sbin/rsyslogd -n -iNONE
StandardOutput=null
Restart=on-failure

# Increase the default a bit in order to allow many simultaneous
# files to be monitored, we might need a lot of fds.
LimitNOFILE=16384

[Install]
WantedBy=multi-user.target
Alias=syslog.service
```

Onder het kopje **Unit** vind je een beschrijving van de service en wat deze nodig heeft om te kunnen werken. Bij *Requires* staat dat **syslog** een `syslog.socket` nodig heeft. Voor nu gaan we ervan uit dat het systeem dit oplost. Wij gaan verder naar de **Service** kop.

Bij *ExecStart* staat het commando dat gebruikt moet worden op de service op te starten. Zoals je kan zien is er geen commando gegeven om de service te stoppen. Een Unix-achtig systeem heeft de `kill` functie om om te gaan met processen (zie Linux CLI documentatie). Deze kan gebruikt worden door **systemd** om processen te stoppen. De *Type* met als waarde `Notify` zegt dat

de daemon aan systemd laat weten wanneer het klaar is met opstarten. Dit heeft als voordeel dat als **systemd** dit signaal krijgt dat het door kan met andere zaken te doen en niet langer hoeft te wachten. Tot slot is er in deze sectie nog de *Restart* optie die we willen behandelen. Het geeft niet aan wat we moeten doen bij een restart, maar wanneer we moeten restarten. In dit geval als **rsyslogd** crashed of elke andere willekeurig melding van een error geeft dan wordt het proces geherstart.

Alle Unit-files zijn platte tekst bestanden en kunnen dus met een willekeurige editor aangepast worden. Als je een bestand (service) hebt gewijzigd of toegevoegd moet je **systemctl** vertellen dat er iets nieuws is. Dat doe je met het volgende commando:

```
$ sudo systemctl daemon-reload
```

Hierna kan je de nieuwe of aangepaste service stoppen, starten of herstarten met de gewijzigde settings.

2.3 systemctl

Na het opstarten van een systeem kan het soms nodig zijn met bestaande processen te stoppen of herstarten omdat er bijvoorbeeld een wijziging in de configuratie is gebeurd, of omdat bepaalde functionaliteit niet langer via deze machine aangeboden wordt. Het kan ook voorkomen dat nieuwe functionaliteit is toegevoegd en dat er een nieuw proces gestart moet worden. Het is dus van belang dat we **systemd** kunnen vertellen om deze acties uit te voeren. Het commando dat doorvoor beschikbaar is heet **systemctl**. **systemctl** kent een aantal standaard commando's die we kunnen meegeven. Voor een uitleg van de syntax van het commando verwijzen we naar de man-page. Wij behandelen hier alleen de belangrijkste commando's:

start Start een service

stop Stop een service

restart Stop een service en als dat klaar is start het weer

reload Vraag een service om zijn configuratie opnieuw te laden, kan niet met alle daemons. Dit is niet de Unit-configuratie, maar de configuratie van de daemon zelf.

Om wat ervaring op te doen met **systemctl** gaan we wat oefenen met een daemon die standaard met de installatie is meegekomen en waar we wat mee kunnen spelen zonder dat het meteen destructief is voor het systeem. De

standaard met de installatie meegekomen daemon is de `systemd-timesyncd` daemon. We gaan deze daemon gebruiken om een beetje vertrouwd te raken met `systemctl`.

```
sudo systemctl status systemd-timesyncd
```

```
● systemd-timesyncd.service - Network Time Synchronization
   Loaded: loaded (/lib/systemd/system/systemd-timesyncd.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2023-02-02 08:21:48 CET; 1 weeks 6 days ago
     Docs: man:systemd-timesyncd.service(8)
  Main PID: 694 (systemd-timesyn)
    Status: "Initial synchronization to time server 205.233.73.201:123 (0.debian.pool.ntp.org)."
```

Tasks:	2 (limit: 38007)
Memory:	1.1M
CPU:	2.039s

```
   CGroup: /system.slice/systemd-timesyncd.service
           └─694 /lib/systemd/systemd-timesyncd

Feb 02 08:21:48 hcl01 systemd[1]: Started Network Time Synchronization.
Feb 02 08:23:18 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 95.179.131.82:123 (2.debian.pool.ntp.org).
Feb 06 08:58:11 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 5.39.184.5:123 (0.debian.pool.ntp.org).
Feb 07 09:00:33 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 74.6.168.73:123 (0.debian.pool.ntp.org).
Feb 07 18:52:41 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 89.208.105.58:123 (0.debian.pool.ntp.org).
Feb 08 08:41:03 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 192.48.105.15:123 (0.debian.pool.ntp.org).
Feb 09 17:53:27 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 94.198.159.16:123 (0.debian.pool.ntp.org).
Feb 13 10:09:09 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 69.89.207.199:123 (0.debian.pool.ntp.org).
Feb 14 09:03:37 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 72.30.35.89:123 (0.debian.pool.ntp.org).
Feb 15 08:47:55 hcl01 systemd-timesyncd[694]: Initial synchronization to time server 205.233.73.201:123 (0.debian.pool.ntp.org).
lines 1-22/22 (END)
```

Figuur 2.1: Status output van `systemctl`

Als je net als in het voorbeeld (Figuur 2.1) een lijn hebt met END dan kan je gebruik maken van 'q' om weer op de command-prompt terecht te komen.

Door aan `systemctl` de commando's stop of start mee te geven kunnen we daemons op ons systeem stoppen en starten. De tijd synchronisatie daemon is op dit moment gestart, dus het eerste wat we kunnen doen is hem stoppen:

```
sudo systemctl stop systemd-timesyncd
```

Met `systemctl status` kunnen nu de status van de daemon zien en dan zien we dat deze gestopt is. Het opnieuw opstarten doen we met start:

```
sudo systemctl start systemd-timesyncd
```

Probeer nu zelf de herstart en reload functies uit. Wat neem je waar?

Om te weten wat `systemctl` allemaal geladen heeft is er het commando:

```
sudo systemctl list-units
```

Hiermee kan je zien wat er allemaal gedaan is door systemd en met `grep` kan je daar de services uit halen die op dit moment actief zijn:

```
sudo systemctl list-units | grep \.service.*running
```

Hoofdstuk 3

Job scheduling

De job scheduler op Unix-achtige besturingssystemen heet **cron**, van chronos (Grieks voor tijd). Een job scheduler zorgt ervoor dat commando's, scripts of taken op een vooraf bepaald moment worden uitgevoerd. Meestal zijn dit taken die regelmatig, bijvoorbeeld 1x per week, uitgevoerd moeten worden. De taken die worden uitgevoerd moeten het doen zonder input van een gebruiker. De output zal weggeschreven moeten worden in een bestand of een log.

3.1 cron

De job scheduler **cron** kan taken uitvoeren per gebruiker, maar kan ook centraal aangestuurd worden via een configuratie bestand dat **/etc/crontab** heet. De meeste **crontab** bestanden hebben een uitvoerige uitleg in het bestand staan met hoe je een regel in **crontab** moet zetten. Deze uitleg ziet er zo uit:

```
# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7)
# | | | | | OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
```

Een crontab regel beschrijft in velden gescheiden door een stuk wit (spatie of tab) wanneer welke taak uitgevoerd moet worden. De velden lopen van links naar rechts, beginnend met op welke minuut er iets moet gebeuren naar uren, dag van de maand, de maand en de dag van de week.

Per veld kunnen er verschillende waardes worden opgegeven. De asterisk (*) betekent elke mogelijke waarde of niet van betekenis. Zo kunnen we een regel maken met in elk veld een asterisk en bij “day of week” kunnen we dan “mon” opgeven. Het opgegeven commando zal dan elke maandag uitgevoerd worden. De velden mogen ook een reeks of lijst bevatten, waarbij de elementen van een lijst gescheiden zijn door komma’s. Zo kunnen we voor “hours” bijvoorbeeld opgeven “8,9,10,11,12,13,14,15,16,17” om iets alleen tijdens de werkuren uit te laten voeren. Voor deze reeks is het makkelijker om het op te geven als “8-17”. Een reeks wordt dus weergegeven met een minteken (hyphen). Als je iets om de 5 minuten wil laten gebeuren dan kan je in het “minute” veld het volgende opgeven “*/5”. En zo kunnen we eindeloze combinaties maken en exact bepalen wanneer een bepaald commando uitgevoerd moet worden.

Om de veiligheid van het systeem te verhogen en niet elk commando door root te laten uitvoeren moet je na de definitie van het moment opgeven welke gebruiker (user-name) het commando moet uitvoeren en pas daarna geef je het commando op dat moet worden uitgevoerd met de eventuele opties en argumenten.

Gebruikers mogen ook zelf scripts of commando’s laten uitvoeren. Hiervoor is er het `crontab` commando. Het `crontab` commando pas niet het centrale `crontab`-bestand aan, maar schrijft deze per gebruiker weg in de directory `/var/spool/cron/crontabs/`. Het is niet de bedoeling dat je bestanden in deze directory met een editor aanpast. Je moet altijd het `crontab` commando gebruiken voor een wijziging.

3.2 at

Hoofdstuk 4

Monitoring services

In de sectie over `journalctl` hebben we gezien hoe we informatie over een proces kunnen opvragen. We kunnen zien of een daemon gestart of gestopt is. Daemons zijn processen die op de achtergrond draaien en die allerlei taken kunnen uitvoeren. Zo is `cron` een daemon, maar een daemon kan ook een mail-server of web-server zijn. Deze laatste twee luisteren op het netwerk naar inkomende verbindingen. De mail-server doet dat voor inkomende mail op poort 25 en de web-server op poort 80. Het is handig als we kunnen zien welke processen er draaien en wat ze aan het doen zijn. Dit hoofdstuk beschrijft enkele commando's die we daarvoor kunnen gebruiken.

4.1 Proces monitoring

Met `ps` kan je zien welke processen er draaien op een systeem. In deze sectie gaan we `ps` met andere opties gebruiken dan we eerder hebben gedaan. De Linux variant van `ps` kent de mogelijkheid om ook opties te gebruiken zonder gebruik van het min teken voor de optie, dit is de zogenaamde BSD-style:

```
$ ps aux
```

doet bijna hetzelfde als `ps -ef`. De output ziet er ook bijna hetzelfde uit:

```
$ ps aux
USER  PID  %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root    1   0.0  0.0 167316 11924 ?        Ss   Feb02   0:48 /sbin/init
root    2   0.0  0.0     0     0 ?        S    Feb02   0:00 [kthreadd]
root    3   0.0  0.0     0     0 ?        I<   Feb02   0:00 [rcu_gp]
root    4   0.0  0.0     0     0 ?        I<   Feb02   0:00 [rcu_par_gp]
root    9   0.0  0.0     0     0 ?        I<   Feb02   0:00 [mm_percpu_wq]
root   10   0.0  0.0     0     0 ?        S    Feb02   0:00
[rcu_tasks_rude_]
```

root	11	0.0	0.0	0	0	?	S	Feb02	0:00	
[rcu_tasks_trace]										
root	12	0.0	0.0	0	0	?	S	Feb02	1:25	[ksoftirqd/0]
root	13	0.0	0.0	0	0	?	I	Feb02	6:07	[rcu_sched]
root	14	0.0	0.0	0	0	?	S	Feb02	0:03	[migration/0]
root	16	0.0	0.0	0	0	?	S	Feb02	0:00	[cpuhp/0]
root	17	0.0	0.0	0	0	?	S	Feb02	0:00	[cpuhp/1]
root	18	0.0	0.0	0	0	?	S	Feb02	0:03	[migration/1]
root	19	0.0	0.0	0	0	?	S	Feb02	0:10	[ksoftirqd/1]

De extra informatie is voornamelijk de extra data over het gebruik van de CPU en van het geheugen (MEM). Deze informatie kennen we ook uit de `top` applicatie. Ook dit commando is behandeld in de 'Linux CLI' handleiding.

4.1.1 `top`

Als een systeem langzaam aan trager aan het worden is willen we graag weten wat er aan de hand is. Met een lijst zoals door `ps` gegeven kunnen we dat lastig monitoren, omdat we niet kunnen zien wat er in de tijd gebeurt. Met `top` kunnen we dat wel. `top` laat je in real-time zien wat er op je systeem gebeurt. Als je `top` opstart ziet het scherm er ongeveer uit zoals in [figuur 4.1](#).

```

top - 11:02:18 up 2 days, 23:34, 1 user, load average: 0.39, 0.35, 0.33
Tasks: 317 total, 4 running, 313 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.9 us, 0.7 sy, 0.0 ni, 96.3 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 31775.7 total, 13797.0 free, 7984.9 used, 9993.8 buff/cache
MiB Swap: 14879.4 total, 14879.4 free, 0.0 used, 22077.7 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR  S  %CPU  %MEM     TIME+ COMMAND
 1919 dennis    20   0  4279456  883352  340912 S   8.0   2.7   396:00.15 firefox-esr
 2927 dennis    20   0 4062088  871328 163144 R   8.0   2.7  181:27.76 Web Content
 2977 dennis    20   0 3328488  446296 204552 S   2.3   1.4   146:00.07 Web Content
20670 dennis    20   0 2821700  460148  98496 S   1.7   1.4    25:43.66 teams
 1987 dennis    9  -11 2486280  27836  21852 S   1.3   0.1   53:41.40 pulseaudio
 3110 dennis    20   0 3269368  393728 191920 S   1.3   1.2   23:26.71 Web Content
  790 root      20   0  874332  19576  13472 S   0.7   0.1    0:58.09 NetworkManager
  941 root      20   0  786396  385148 318256 S   0.7   1.2   53:34.86 Xorg
 1916 dennis    20   0 3458804  830680 205320 S   0.7   2.6   33:12.11 thunderbird
 2909 dennis    20   0 3565536  550048 227900 S   0.7   1.7   38:04.83 Web Content
  581 root     -51   0      0      0      0 S   0.3   0.0    2:38.58 irq/31-rm14_smb
  788 message+ 20   0  10644   5644   3776 S   0.3   0.0    0:23.43 dbus-daemon
 1930 dennis    20   0  513596  57068  33708 S   0.3   0.2    4:35.38 xfce4-terminal
 2139 dennis    20   0  369644  34540  26920 S   0.3   0.1    0:29.36 nm-applet
 2949 dennis    20   0 3180716  456052 133832 S   0.3   1.4   12:51.42 Web Content
 3009 dennis    20   0 3744508  1.0g 215844 R   0.3   3.2  212:17.05 Web Content
 4094 dennis    20   0 2965588  317884 179028 R   0.3   1.0    6:52.13 Web Content
19111 dennis    20   0 1111244  31260  22384 S   0.3   0.1    7:46.68 VBoxSVC
22429 dennis    20   0 2119796 200600  56600 S   0.3   0.6    0:05.21 gimp-2.10
23047 root      20   0      0      0      0 I   0.3   0.0    0:00.41 kworker/0:4-events_power_efficie+
23413 dennis    20   0  11304   3868   3132 R   0.3   0.0    0:00.11 top
   1 root      20   0 171164  11036  7920 S   0.0   0.0    0:05.80 systemd
   2 root      20   0      0      0      0 S   0.0   0.0    0:00.10 kthreadd
   3 root      0 -20      0      0      0 I   0.0   0.0    0:00.00 rcu_gp
   4 root      0 -20      0      0      0 I   0.0   0.0    0:00.00 rcu_par_gp
   6 root      0 -20      0      0      0 I   0.0   0.0    0:00.00 kworker/0:0H-kblockd
   8 root      0 -20      0      0      0 I   0.0   0.0    0:00.00 mm_percpu_wq
   9 root      20   0      0      0      0 S   0.0   0.0    0:12.48 ksoftirqd/0
  10 root      20   0      0      0      0 I   0.0   0.0    0:38.01 rcu_sched
  11 root      20   0      0      0      0 I   0.0   0.0    0:00.00 rcu_bh
  12 root      rt   0      0      0      0 S   0.0   0.0    0:00.68 migration/0
  14 root      20   0      0      0      0 S   0.0   0.0    0:00.00 cpuhp/0
  15 root      20   0      0      0      0 S   0.0   0.0    0:00.00 cpuhp/1
  16 root      rt   0      0      0      0 S   0.0   0.0    0:00.96 migration/1
  17 root      20   0      0      0      0 S   0.0   0.0    0:01.30 ksoftirqd/1
  19 root      0 -20      0      0      0 I   0.0   0.0    0:00.00 kworker/1:0H-kblockd
  20 root      20   0      0      0      0 S   0.0   0.0    0:00.00 cpuhp/2
  21 root      rt   0      0      0      0 S   0.0   0.0    0:01.06 migration/2
  22 root      20   0      0      0      0 S   0.0   0.0    0:07.17 ksoftirqd/2

```

Figuur 4.1: Top

Met `q` verlaten we `top`.

`top` geeft je in één scherm heel veel informatie over je systeem en welke processen het meeste van je systeem vragen. Standaard sorteert `top` de processen op processor gebruik. We wie in het lijstje dat `firefox-esr` 8.0% CPU gebruikt en 2.7% van het geheugen. Met `M` en `P` (**let op** hoofdletters), kan je schakelen tussen een overzicht van het meeste processor (`P`) gebruik of het meeste memory (`M`) gebruik.

4.2 Daemons en netwerken

Veel daemons worden gebruikt om diensten aan het netwerk aan te bieden. Om een dienst aan te kunnen bieden moet een daemon luisteren op een port

en zodra er een verbinding wordt gemaakt op de port moet de daemon een nieuw proces laten luisteren. Veel daemons kan je vertellen op welke interface of welk IP-adres ze moeten luisteren, zo kan je er bijvoorbeeld voor zorgen dan een MySQL server alleen luistert op de interface van localhost zodat hij niet van buitenaf beschikbaar is.

Als we een daemon opstarten willen we kunnen controleren of de daemon daadwerkelijk luistert op een interface. Van oudsher werd hiervoor het `netstat` commando gebruikt, nieuwere systemen hebben meestal het `ss` commando. Beide commando's hebben de dezelfde functionaliteit. Je kan ze gebruiken om te zien welk proces er luistert op welke interface en je kan zien welke verbindingen er op dit moment gaande zijn. We zullen voor de compleetheid beide commando's laten zien.

Als een daemon luistert op een poort en we willen kunnen zien of dat daadwerkelijk gebeurt dan moeten we de commando's vragen om ons de LISTENING status te laten zien:

```
$ netstat -l
```

```
$ ss -l
```

Als je beide commando's geïnstalleerd hebt staan dan zal je zien de de output iets verschillend is.

Er luisteren over het algemeen veel processen naar allerlei verschillende zaken. Vaak willen we alleen weten of ze op TCP of UDP luisteren. Met de `-u` optie kunnen we kiezen voor UDP en met `-t` voor TCP. We kunnen ze ook tesamen gebruiken om zowel de TCP als de UDP porten te laten zien:

```
$ netstat -lut
```

```
$ ss -lut
```

Je ziet dat voor beide commando's de opties identiek zijn.

Tot slot van het bekijken van de luisterende poorten willen we ook kunnen zien welk proces (welke daemon) de poort gebruikt. Dit kan alleen root zien, dus moeten we `sudo` gebruiken:

```
$ sudo netstat -tulp
```

```
$ sudo ss -tulp
```

Beide commando's kunnen ook gebruikt worden om te zien welke verbindingen er nu open staan met je computer:

```
$ sudo netstat -tuanp | grep -vE 'LISTEN|:*\'
```

```
$ sudo ss -tu -o state connected
```

Gebruik de man-page van de verschillende commando's om te zien wat de opties betekenen.

Index

/boot/, 3

commando
 top, 12

cron, 9

crontab, 9

init, 5

job scheduler, 9

kernel, 2

modules, 2

netstat, 14

ps, 11

scheduler, 9

ss, 14

systemctl, 7

systemd, 5
 timesyncd, 8

timesyncd, 8

top, 12

vmlinuz, 3